

The ECU Sees Only Signals

Script: Chapter 1 (The ECU Sees Only Signals)

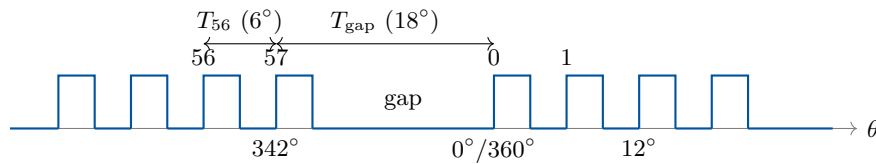
with solutions

Learning goals

- Classify the engine sensor signals (continuous/discrete time, analog/binary amplitude, deterministic/stochastic, periodic/apperiodic) and know their characteristic time scales.
- *Bonus (optional)*: describe sensor signals quantitatively: mean, RMS, extreme values, autocorrelation as a numerical test for periodicity.
- Extract a first piece of information from a raw signal: crank angle and engine speed from the 60-2 tooth timing, including the effect of the timer resolution.
- Synchronise crank and cam signal to determine the 720° phase of the engine cycle (cylinder identification).

Background: the 60-2 trigger wheel

The crank wheel of the virtual engine has 60 tooth positions (6° each), two of which are missing. The Hall sensor delivers a binary signal; the rising edge of tooth $k = 0, \dots, 57$ occurs at $\theta = 6k^\circ$ (modulo 360°), the reference mark is the first rising edge after the gap.

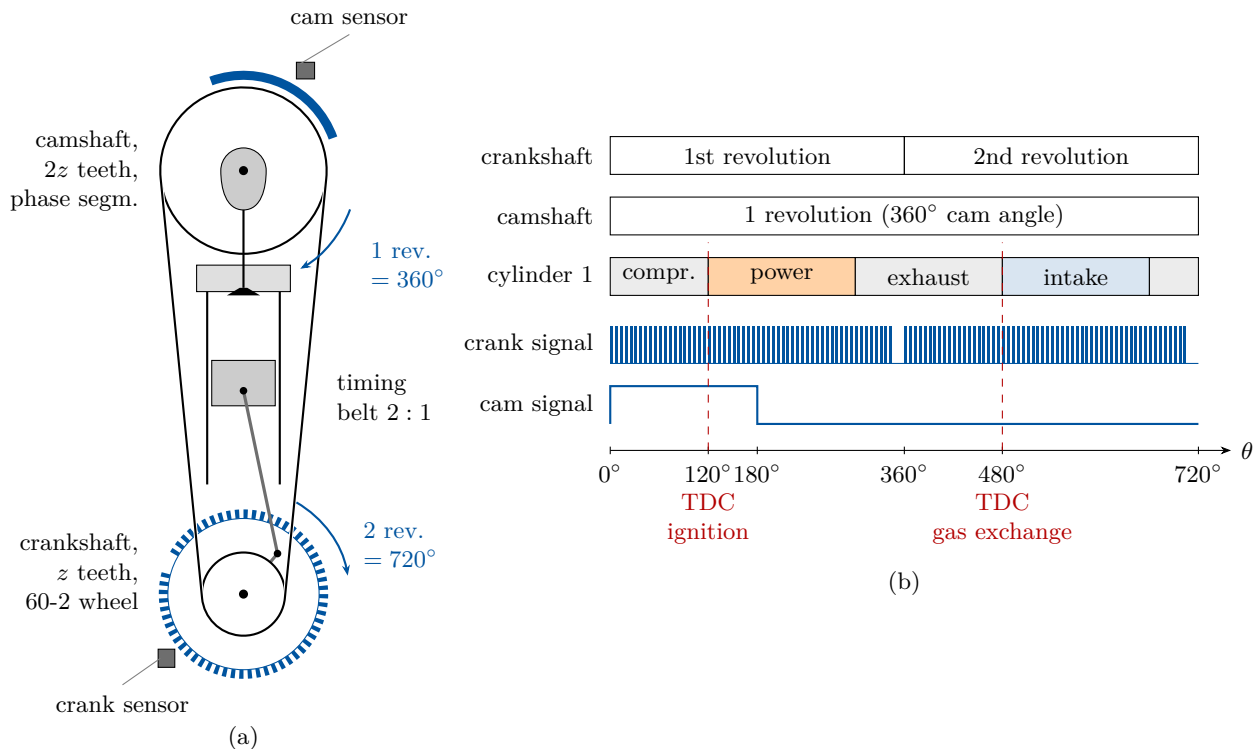


The tooth period T_k is measured from rising edge to rising edge. Because one tooth corresponds to $6^\circ = 1/60$ revolution, the engine speed in rpm is

$$n = \frac{60 \text{ s/min}}{60 T_k} = \frac{1}{T_k/\text{s}} \text{ rpm}, \quad n = \frac{3}{T_{\text{gap}}/\text{s}} \text{ rpm (gap)}, \quad n = \frac{60}{T_{\text{rev}}/\text{s}} \text{ rpm (revolution)}. \quad (1)$$

Background: crankshaft, camshaft and the 720° cycle

A four-stroke engine needs *two* crankshaft revolutions for one working cycle (intake, compression, power, exhaust), but every valve opens only once per cycle. The camshaft is therefore driven by a timing belt (or chain) with a ratio of 2 : 1: its sprocket has twice as many teeth as the crankshaft sprocket, so it turns once while the crankshaft turns twice. The crank angle θ is counted over the full cycle, $0^\circ \leq \theta < 720^\circ$, which is exactly one camshaft revolution (360° cam angle). The 60-2 wheel on the crankshaft repeats after 360° crank angle, the single segment of the cam phase wheel only after 720°.



(a) Schematic front view of one cylinder with crank drive and overhead camshaft; the timing belt turns the camshaft once per two crankshaft revolutions. (b) One working cycle of cylinder 1 over the crank angle: the crank wheel pattern and the piston position repeat after 360°, the strokes and the cam signal only after 720°.

Exercise 1.1 Signal classification and time scales

PAPER

- Classify the five engine signals of Table 1.2 of the script (crank, cam, knock, ion current, lambda) with respect to (i) time: continuous/discrete, (ii) amplitude: analog/binary (digital), (iii) deterministic/stochastic, (iv) periodic/quasi-periodic/apperiodic. Justify every entry in one line. Note that the answer may differ for the physical sensor output and for the representation inside the ECU.
- Discuss the borderline cases: (i) Is the crank signal periodic during an acceleration? Is it periodic in another independent variable? (ii) Why does the script call the knock signal *quasi-periodic*, although a single knock event is a transient? Introduce the notion of a signal whose *statistics* are periodic in the crank angle.
- Fill in the following table for an engine speed of 3000 rpm: crank revolution period, tooth period and tooth frequency, duration of the gap (rising to rising edge), cam period, cam HIGH time, interval between two combustions (firing frequency), duration of 1° crank angle, time window of a knock onset (5°–25° after TDC), number of 6.5 kHz knock oscillations within one decay time $\tau = 0.8$ ms.
- Crank and cam: the crankshaft turns 720° per engine cycle, the camshaft 360°. Why is the crank signal alone not sufficient to decide whether the piston of cylinder 1 is in its compression or in its exhaust stroke? After switching on the ignition with the engine cranking at 200 rpm, what is the worst-case time until the ECU knows (i) the crank angle modulo 360°, (ii) the full 720° phase, if the cam level is evaluated at $\theta = 60^\circ$ after the reference mark?

Solution

a) Classification (physical signal $\rightarrow$ inside the ECU):

Signal	time	amplitude	predictability	periodicity
crank	CT $\rightarrow$ event times t_k	binary (Hall)	deterministic up to small speed fluctuations	periodic in θ (360°); in t only at constant speed
cam	CT $\rightarrow$ level sampled at tooth events	binary	deterministic	periodic in θ (720°)
knock	CT $\rightarrow$ DT after ADC	analog $\rightarrow$ quantised	stochastic (noise, random knock onset, amplitude, phase)	quasi-periodic: deterministic parts repeat every 180° , knock is random
ion current	CT $\rightarrow$ DT	analog	partly deterministic (shape of the peaks), cycle-to-cycle variation	aperiodic per event; event pattern repeats per 180°
lambda	CT $\rightarrow$ DT (slow, 10–100 Hz)	analog, but practically two-valued (0.1/0.9 V)	slowly varying, disturbed	aperiodic; in the closed loop a limit cycle (quasi-periodic, ≈ 1 Hz)

Crank and cam are physically continuous-time voltages with two levels; the ECU, however, does not sample them but time-stamps their edges with a capture timer – the information is in the *sequence of edge times* t_k , i.e. a discrete-time signal indexed by the tooth number. All analog signals become discrete-time and quantised (= digital) after the ADC.

b) (i) During an acceleration the tooth period changes from tooth to tooth, so $x(t) = x(t + T)$ holds for no T : the crank signal is not periodic in time. It is, however, *exactly* periodic in the crank angle, $x_c(\theta) = x_c(\theta + 360^\circ)$, independently of the speed – the reason why many engine algorithms work in the angle domain (Sheet 4). (ii) Each knock burst is a transient (damped oscillation), its onset, amplitude and phase are random and it does not occur in every cycle. But the *probability* of a burst, the valve impacts, the combustion vibration and the noise level repeat every 180° (each TDC): the mean value and the autocorrelation of the signal are periodic functions of the crank angle. Such a signal is called *cyclostationary*. “Quasi-periodic” in the script means exactly this.

c) At $n = 3000$ rpm (50 rev/s, $\omega = 18\,000^\circ/\text{s}$):

quantity	formula	value
crank revolution	$60/n$	20 ms (50 Hz)
tooth period / frequency	$T_{\text{rev}}/60$	333 μs / 3 kHz
gap (rising to rising)	$3 T_z$	1 ms
engine cycle = cam period	$2 T_{\text{rev}}$	40 ms (25 Hz)
cam HIGH time	180°	10 ms
combustion interval	$720^\circ/4 = 180^\circ$	10 ms (firing frequency 100 Hz = 2nd order)
1° crank angle	$1/\omega$	55.6 μs
knock onset window	$5^\circ \dots 25^\circ$ ATDC	0.28 ms $\dots$ 1.39 ms after TDC
knock oscillations in τ	$f_1 \tau$	$6.5 \cdot 0.8 = 5.2$

The signals thus span four decades of time scales: knock (100 μs), teeth (300 μs), combustions (10 ms), lambda (1 s).

d) The crank wheel looks identical in both revolutions of a cycle: at $\theta = 120^\circ$ (compression TDC of cylinder 1, ignition) and at $\theta = 480^\circ$ (gas-exchange TDC of cylinder 1, overlap) the crank signal is the same. Only the camshaft, which turns once per cycle, distinguishes the two revolutions. At 200 rpm one revolution lasts 300 ms. (i) Worst case: the ECU starts just after the reference mark and must wait for the next gap *and* needs one normal tooth period

before it to apply the ratio test: almost one revolution, ≈ 300 ms. (ii) Then another 60° to the cam evaluation: $420^\circ \approx 350$ ms. Before that, the ECU can only use “wasted spark” (fire both cylinders 1 and 4 at every crank TDC) and must not inject sequentially.

Exercise 1.2 60-2 crank wheel and engine speed

WOKWI

Open the Wokwi project `code/wokwi/sheet01_crank/` (Arduino Uno, engine simulator chip, logic analyzer, two LEDs, push button). `eng:CRANK` is connected to pin 2 (external interrupt INT0), `eng:CAM` to pin 4. The ISR `crank_isr()` is called at every rising crank edge and time-stamps it. With `TIMEBASE_TIMER1 = 1` the time stamp is the 16-bit Timer1 counter (prescaler 8, $0.5\mu\text{s}$ per tick), with 0 it is `micros()` (resolution $4\mu\text{s}$ on the Uno).

- Preparation (paper).* Derive (1). A timer of resolution Δ quantises the tooth period. Show that the resulting quantisation step of the per-tooth speed is $\Delta n \approx n^2 \Delta$ (with n in rpm and Δ in s) and evaluate it for $\Delta = 4\mu\text{s}$ and $0.5\mu\text{s}$ at 600, 3000 and 7000 rpm. Compare with the amplitude of the 1% speed ripple of the model. The gap is detected by the ratio test $T_k > 2T_{k-1}$: estimate the angular acceleration (in rpm/s) at which the test would fail (missed gap, or false gap), and compare with realistic engine accelerations ($\lesssim 10\,000$ rpm/s).
- Complete the ISR: gap detection, tooth counter, per-revolution statistics, TDC LED (pin 12) on for 30° starting at each TDC edge. Which tooth numbers belong to $\theta = 120^\circ$ and 300° ? Check with the logic analyzer (D0 = crank, D2 = TDC output of the chip, D3 = your LED) and report the column `teeth` and `sync_err`.
- Complete the speed computation in `loop()` (per revolution, last tooth, min/max per tooth). At 3000 rpm, compare `rpm_rev` with the spread `rpm_min...rpm_max` for both time bases. Press the button to dump 120 tooth periods, copy only this block and plot the per-tooth speed (column `rpm`), e.g. in a spreadsheet or with your CSV plot program of Bonus Exercise 0.B1 d) in JSLinux. Which part of the “jitter” is information, which part is measurement error?
- Move the speed slider (600...7000 rpm) and repeat c) at 7000 rpm. Why is printing one CSV line per *tooth* not an option (estimate the UART load)?

Solution

a) One tooth = $6^\circ = \frac{1}{60}$ rev, so the speed is $\frac{1}{60T_k}$ rev/s = $\frac{60}{60T_k}$ rpm = $1/T_k$ (in rpm with T_k in s). The gap covers 18° , a revolution 360° , hence $3/T_{\text{gap}}$ and $60/T_{\text{rev}}$. With $n = 1/T$: $\left| \frac{dn}{dT} \right| = \frac{1}{T^2} = n^2$, so a period error of one tick changes n by $\Delta n \approx n^2 \Delta$:

n / rpm	T_k	Δn ($\Delta = 4\mu\text{s}$)	Δn ($\Delta = 0.5\mu\text{s}$)	ripple ($\pm 1\%$)
600	$1667\mu\text{s}$	1.4 rpm	0.2 rpm	± 6 rpm
3000	$333\mu\text{s}$	36 rpm	4.5 rpm	± 30 rpm
7000	$143\mu\text{s}$	196 rpm	24.5 rpm	± 70 rpm

The relative resolution $\Delta n/n = \Delta/T_k$ degrades linearly with speed. With `micros()` the ripple is buried in the quantisation above ≈ 2500 rpm; with the $0.5\mu\text{s}$ timer it is resolved over the whole range. Real ECUs use capture timers with ≤ 100 ns resolution.

Gap test: let ω_1 be the speed during the tooth before the gap, ω_2 the mean speed across the gap. Then $T_{\text{gap}}/T_{\text{prev}} = 3\omega_1/\omega_2 < 2$ requires $\omega_2 > 1.5\omega_1$ within about two tooth periods ($\approx 12^\circ/\omega_1$): $\dot{\omega} \approx 0.5\omega_1^2/12^\circ$, i.e. $\dot{n} \approx n^2/4$ (rpm/s, n in rpm) – $90\,000$ rpm/s at 600 rpm. A false gap ($T_k/T_{k-1} > 2$ for a normal tooth) needs the speed to halve within one tooth, $\dot{n} \approx -n^2/2$, $-180\,000$ rpm/s at 600 rpm. Both are an order of magnitude beyond real engines, so threshold 2 (the geometric mean of 1 and 3 would be 1.73) is robust in normal operation. Critical is only the start (cranking at 200 rpm with $\pm 30\%$ speed fluctuations from the compression strokes, limit $\dot{n} \approx 10\,000$ rpm/s), where ECUs add plausibility checks (tooth count 58 between gaps, cam edge

position).

b) Key part of the solution (code/solutions/wokwi/sheet01_crank/sketch.ino):

```

1 void crank_isr(void)
2 {
3     tick_t now = now_ticks();
4     tick_t T = now - t_last;          /* unsigned: wrap-around safe */
5     t_last = now;
6     if (!have_edge) { have_edge = true; return; }
7
8     bool gap = (T_prev != 0) && (T > 2 * T_prev);
9     T_prev = T;
10
11     if (gap) {
12         if (syncd && tooth != 57) sync_errors++;
13         /* revolution complete: hand over statistics to loop() */
14         rev_ticks = rev_acc + T;
15         Tmin_rev = Tmin_acc; Tmax_rev = Tmax_acc;
16         teeth_rev = teeth_acc + 1;      /* edges incl. this one */
17         rev_ready = syncd;
18         rev_acc = 0; teeth_acc = 0; Tmin_acc = 0xFFFF; Tmax_acc = 0;
19         syncd = true;
20         tooth = 0;
21         if (phase >= 0) phase = 360 - phase; /* next revolution */
22     } else {
23         if (!syncd) return;
24         tooth++;
25         rev_acc += T;
26         teeth_acc++;
27         if (T < Tmin_acc) Tmin_acc = T;
28         if (T > Tmax_acc) Tmax_acc = T;
29         T_tooth_rev = T;
30     }
31
32     /* crank angle of this edge = phase + 6 * tooth */
33     if (tooth == TDC_TOOTH_A || tooth == TDC_TOOTH_B) {
34         digitalWrite(TDC_LED, HIGH);
35         /* cylinder 1: TDC at 120 deg, only known with cam phase */
36         if (phase == 0 && tooth == TDC_TOOTH_A) digitalWrite(CYL1_LED, HIGH);
37     }
38     if (tooth == TDC_TOOTH_A + LED_TEETH || tooth == TDC_TOOTH_B + LED_TEETH) {
39         digitalWrite(TDC_LED, LOW);
40         digitalWrite(CYL1_LED, LOW);
41     }
42
43     /* Ex 1.4: cam level in the middle of a cam state (60 or 420 deg) */
44     if (tooth == CAM_TOOTH) {
45         int16_t p = digitalRead(CAM_PIN) ? 0 : 360;
46         if (phase >= 0 && p != phase) cam_errors++;
47         phase = p;
48     }

```

120° and 300° correspond to the rising edges of teeth 20 and 50. Every revolution reports **teeth** = 58 edges (57 normal teeth + the gap edge) and **sync_err** = 0. In the logic analyzer the LED rises a few microseconds (interrupt latency) after the rising edge of tooth 20 (50), i.e. practically together with the 50 µs TDC pulse of the chip, which is emitted exactly at 120°/300°. The LED marks the crank-angle *event*; it is lit for 5 teeth = 30°.

c) Expected values at 3000 rpm (obtained with an offline model of exactly this measurement, code/solutions/wokwi/sheet01_crank/tooth_sim.c, over 200 revolutions; in Wokwi rare additional outliers of a few µs can occur when the crank interrupt has to wait for the Timer0 or UART interrupt):

time base	rpm_rev	per-tooth min. . . max	per-tooth std
<code>micros()</code> , 4 μ s	2999.4...3000.0	2941...3049	25.7 rpm
Timer1, 0.5 μ s	2999.85	2967...3030	21.7 rpm

The per-revolution speed is very precise (one tick on 20 ms: $\Delta n/n = 0.5/40000$) and slightly *below* 3000, because the mean of $1/(1 + \varepsilon \sin 2\theta)$ is $1 + \varepsilon^2/2$ ($n\varepsilon^2/2 = 0.15$ rpm). The per-tooth values with Timer1 show the *real* speed fluctuation: $\pm 1\%$, two periods per revolution, std $30/\sqrt{2} = 21.2$ rpm – this is information (in a real engine the torque pulses of the individual combustions; used e.g. for misfire detection). With `micros()` the values jump in steps of ≈ 36 rpm (332, 336 μ s) – a measurement error of the same size as the information. The plotted dump shows a clean sine with 2 periods per 58 teeth for Timer1 and a staircase-like, noisy version for `micros()`.

d) At 7000 rpm the model gives per-tooth std 94 rpm (`micros`, range 6757...7143) vs. 51 rpm (Timer1, 6920...7092, ripple ± 70 rpm); the per-revolution value stays within ± 3 rpm resp. 0.5 rpm. When the slider is moved, the speed changes abruptly in the simulator; the gap is still found (ratio 3 vs. 1, see a)) as long as the jump does not fall exactly into the gap. UART: at 7000 rpm there are $7000 \cdot 58/60 \approx 6800$ teeth/s; a line such as 1234,57,286,6993.0 has ≈ 20 characters, i.e. 136 000 chars/s – twelve times the capacity of 115200 baud (11 520 chars/s). Hence statistics per revolution and block dumps from RAM.

Exercise 1.3 Cam/crank synchronisation

CHALLENGE

Extend your sketch such that it determines the 720° phase from the cam signal (`eng:CAM`, HIGH for $0^\circ \leq \theta < 180^\circ$).

- Why should the cam level not be read at the reference edge ($\theta = 0^\circ$), and which tooth is a good choice instead?
- Implement `phase` (0° or 360°), toggle it at every gap, and check it at every revolution (`cam_err`). Light the LED on pin 11 only at the compression TDC of cylinder 1 and verify it with the logic analyzer (D1 = cam, D4 = LED).
- Give the crank angle and the number of the cylinder that fires at each of the four TDC events.

Solution

- The cam edge at $\theta = 0^\circ$ coincides with the reference edge of the crank signal (in the simulator both are written in the same event, on a real engine the cam edge has a tolerance of several degrees and drifts with cam phasing). Reading the level there is a race. The level must be read in the middle of a cam state, e.g. at tooth 10 (60° or 420°): HIGH $\Rightarrow$ first revolution (phase 0°), LOW $\Rightarrow$ second revolution (phase 360°). Any tooth with $\geq 10^\circ$ distance to 0° and 180° works.
- See the listing in Exercise 1.2 (line 21 and lines 43–48): at every gap `phase = 360 - phase`, at tooth 10 the cam level is compared with the predicted phase; `cam_err` stays 0 and the printed `phase` alternates 0, 360, 0, ... The green LED is on only every second time the red TDC LED lights up, i.e. once per engine cycle (every 40 ms at 3000 rpm).
- $\theta = \text{phase} + 6^\circ k$; firing order 1-3-4-2:

phase	tooth	crank angle	cylinder
0°	20	120°	1
0°	50	300°	3
360°	20	480°	4
360°	50	660°	2

Deliverables

1.1: completed classification table and time-scale table. 1.2: sketch, screenshot of the logic analyzer (gap, TDC, LED), table of rpm statistics for both time bases at 3000 and 7000 rpm, plot of a tooth-period dump. 1.3 (optional): extended sketch, logic analyzer screenshot with cam and cylinder-1 LED. Bonus 1.B1 (optional): program, table of statistics and periods, short interpretation.

Bonus: JSLinux (optional)

For interested students: the following exercise analyses all engine signals offline in JSLinux (set up as in Bonus Exercise 0.B1).

Exercise 1.B1 Signal zoo**JS LINUX**

The program `code/bonus/jslinux/sheet01/signal_zoo.c` generates 160 ms (four engine cycles) of the crank, cam, knock and ion signals at 3000 rpm, $f_s = 25$ kHz, and 20 s of a lambda sensor signal ($f_s = 200$ Hz) for a slowly oscillating $\lambda(t)$. Additionally, a crank signal during a speed ramp 2000 rpm to 4000 rpm is generated.

- Complete `stats()`: mean, RMS value, minimum and maximum. Run the program and explain the values of the crank and cam signals analytically. Why is the mean of the knock signal not zero, although the knock sensor is AC coupled?
- The normalised autocorrelation

$$r[m] = \frac{\hat{R}[m]}{\hat{R}[0]}, \quad \hat{R}[m] = \frac{1}{N-m} \sum_{n=0}^{N-1-m} (x[n] - \bar{x})(x[n+m] - \bar{x})$$

is close to 1 at lags m where the signal resembles a shifted copy of itself. Complete `autocorr()` and `period_lag()` (first local maximum after the first zero crossing that reaches at least 90 % of the largest value). Report the estimated period and r for all six signals.

- Interpret the results: Which signals are periodic, which only quasi-periodic? Why does the crank signal show a strong peak already at half a revolution? What happens to the periodicity of the crank signal during the ramp – and which lag is still detected? Why would the $1/N$ -normalised (biased) estimate be misleading here?

Solution

a) Output of the reference solution `code/solutions/bonus/jslinux/sheet01/signal_zoo.c`:

signal	mean	rms	min	max
crank	0.4808	0.6934	0.0000	1.0000
cam	0.2500	0.5000	0.0000	1.0000
knock	0.0433	0.1234	-1.1454	1.5144
ion	0.0993	0.2399	-0.1269	1.0440
lambda	0.5004	0.6195	0.1000	0.9000

For a 0/1 signal with duty cycle d : mean = d , RMS = $\sqrt{d}$. Crank: $d = 58 \cdot 3^\circ / 360^\circ = 0.4833$ (measured 0.4808 because a HIGH phase of $3^\circ = 167 \mu\text{s}$ is only 4.17 samples long, sampling rounds), RMS = $\sqrt{0.4808} = 0.6934$. Cam: $d = 180/720 = 0.25$, RMS = 0.5. The knock signal contains the low-frequency combustion vibration, a positive half sine of amplitude 0.2 V over 60° after every TDC: $0.2 \cdot \frac{2}{\pi} \cdot \frac{60}{180} = 0.042$ V – this is the measured mean. (A real piezo sensor is AC-coupled; the model represents the signal after a high-pass with a cut-off far below the combustion frequency.) The large extremes ($-1.15/+1.51$ V) stem from knock events.

b), c) Result of the period search:

crank 3000	first peak: lag 250 =	10.00 ms r = 0.929	max: lag 500 r = 0.988
crank ramp	first peak: lag 8 =	0.32 ms r = 0.348	max: lag 8 r = 0.348
cam	first peak: lag 1000 =	40.00 ms r = 1.000	max: lag 1000 r = 1.000

knock	first peak: lag	244 =	9.76 ms	$r = 0.418$	max: lag	500	$r = 0.438$
ion	first peak: lag	250 =	10.00 ms	$r = 0.992$	max: lag	250	$r = 0.992$
lambda	first peak: lag	251 =	1255.00 ms	$r = 0.981$	max: lag	500	$r = 0.982$

- *Cam*: exactly periodic, $r = 1.000$ at 40 ms = one engine cycle.
- *Crank*: the true period is one revolution (20 ms, $r = 0.988$; not exactly 1 because of the speed ripple and because the period is 500.03 samples, not an integer). A shift by 180° maps 58 teeth onto teeth except near the gap, therefore $r(250) = 0.929$ is almost as high: the gap, which carries the position information, is only a small part of the signal energy. With the 90% rule the detector reports 180° – a reminder that a period detector must be designed for the feature of interest. (Small peaks at multiples of 8.33 samples come from the teeth.)
- *Crank during the ramp*: the tooth period shrinks from 12.5 to 6.25 samples; no long-lag peak survives; only the tooth-scale peak at lag 8 ($r = 0.35$) remains. In time the signal is not periodic – in the angle domain it would be (Sheet 4).
- *Knock*: $r \approx 0.42$ at ≈ 10 ms (180°): only the deterministic parts (combustion vibration, valve impacts at 8 kHz) repeat; noise and the random knock bursts (random onset and phase) do not correlate. Quasi-periodic, cyclostationary.
- *Ion current*: $r = 0.992$ at 180° – in the model all cylinders produce the same deterministic ion curve; on a real engine cycle-to-cycle variations make each event an individual (aperiodic) transient.
- *Lambda*: period 1.25 s = $1/0.8$ Hz, $r = 0.98$; the switching sensor turns the small λ oscillation into an almost rectangular signal.

The biased estimator ($1/N$ instead of $1/(N - m)$) multiplies $r[m]$ by $(N - m)/N$; at lag 1000 of a 4000-sample record this is a factor 0.75, so the cam signal would show $r = 0.75$ although it is perfectly periodic, and shorter lags would always be favoured. Key part of the solution:

```

1 static void autocorr(const double *x, int n, double *rr, int maxlag)
2 {
3     double mean = 0.0, r0;
4     int i, m;
5     for (i = 0; i < n; i++) mean += x[i];
6     mean /= n;
7     for (m = 0; m <= maxlag; m++) {
8         double s = 0.0;
9         for (i = 0; i + m < n; i++) s += (x[i] - mean) * (x[i + m] - mean);
10        rr[m] = s / (n - m);
11    }
12    r0 = rr[0] > 0.0 ? rr[0] : 1.0;
13    for (m = 0; m <= maxlag; m++) rr[m] /= r0;
14 }
15
16 /* period estimate: after the first zero crossing, find the largest
17 autocorrelation value rmax; the period is the FIRST local maximum that
18 reaches 0.9 rmax (avoids picking a multiple of the period).
19 Returns the lag (0 if none), *peak = r[lag]. */
20 static int period_lag(const double *rr, int maxlag, double *peak)
21 {
22     int m0 = 1, m;
23     double rmax = -1.0;
24     while (m0 <= maxlag && rr[m0] > 0.0) m0++; /* first zero crossing */
25     for (m = m0; m <= maxlag; m++) if (rr[m] > rmax) rmax = rr[m];
26     for (m = m0 + 1; m < maxlag; m++)
27         if (rr[m] >= rr[m - 1] && rr[m] >= rr[m + 1] && rr[m] >= 0.9 * rmax) {
28             *peak = rr[m];
29             return m;

```

```
30     }  
31     *peak = 0.0;  
32     return 0;  
33 }
```